

## REAL-TIME WEB APPLICATION · ASSIGNMENT REPORT

# PulseBoard

A live classroom task board powered by REST APIs, WebSockets and SQLite

## PROJECT SUMMARY

PulseBoard lets classmates create, review and update shared tasks. Persistent changes travel through a JSON REST API into SQLite; authenticated Socket.IO connections distribute each change to the whole classroom without a refresh.

### 01 REST + JSON

GET · POST · PUT · DELETE

### 02 LIVE EVENTS

Socket.IO · shared class room

### 03 DATA + AUTH

SQLite · bcrypt · JWT

## SUBMISSION DETAILS

| Item        | Details                                                                                     |
|-------------|---------------------------------------------------------------------------------------------|
| Assignment  | Real-time web application using REST APIs and WebSockets                                    |
| Application | PulseBoard — collaborative classroom task manager                                           |
| Repository  | <a href="https://github.com/prasanth-selva/WEB_TECH">github.com/prasanth-selva/WEB_TECH</a> |
| Prepared    | 07 October 2026                                                                             |

Demonstration account · [demo@pulseboard.app](mailto:demo@pulseboard.app) / Classroom2026!

# Contents

| Section | Topic                                              |
|---------|----------------------------------------------------|
| 1       | Project overview and requirements                  |
| 2       | System architecture and workflow                   |
| 3       | REST API design and JSON examples                  |
| 4       | Database design and real-time exchange             |
| 5       | Authentication, security and error handling        |
| 6       | Verification results and rubric mapping            |
| 7       | Application screenshots                            |
| 8       | Run instructions, project structure and conclusion |

## 1. Project overview

PulseBoard is a browser-based collaboration tool for a single demonstration classroom. Each student signs in, sees the same shared task board, and can create, update, move or remove tasks. The project deliberately combines a request/response interface for durable data with a push channel for immediate updates.

### Assignment objectives

- Demonstrate client-server communication using REST endpoints and JSON request/response bodies.
- Implement the complete task lifecycle with GET, POST, PUT and DELETE operations backed by SQLite.
- Authenticate HTTP and WebSocket clients, validate input, and return predictable errors.
- Broadcast task changes to separately authenticated classmates in real time.

### Scope decision

The demo represents one shared class, “Class 04 · Web Tech”. Any valid account registered in this teaching instance joins that room and can collaborate on its tasks. The task creator remains visible for attribution. A production product should add explicit classroom membership, role-based permissions and protected multi-tenant room selection.

## 2. System architecture and workflow

The browser uses two complementary channels. Fetch requests carry a bearer JWT to Express and receive JSON responses; a Socket.IO connection authenticated with the same token subscribes to the classroom room. Express writes the task change to SQLite before emitting the corresponding event.

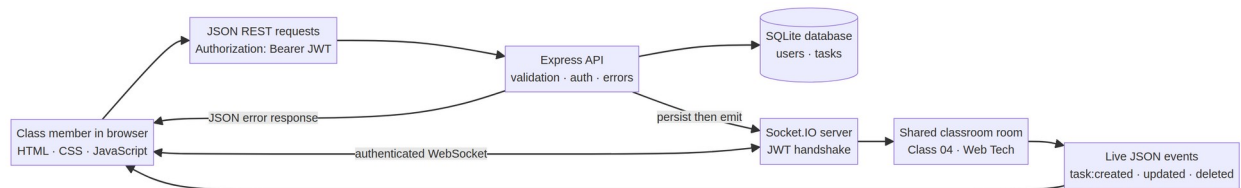


Figure 1. PulseBoard client-server architecture.

| Component      | Responsibility                                                                                           |
|----------------|----------------------------------------------------------------------------------------------------------|
| Browser UI     | Semantic HTML/CSS; fetch-based CRUD; token-bearing Socket.IO connection; board and live activity feed.   |
| Express server | Hosts static assets, authenticates requests, validates fields, returns JSON and centralizes errors.      |
| SQLite         | Persists account and task records; foreign keys, constraints and indexes protect/accelerate data access. |
| Socket.IO      | Authenticates the handshake, joins the shared class room, broadcasts task and presence events.           |

### End-to-end workflow

- Sign in or register: bcrypt verifies or hashes the password; the server returns an eight-hour JWT.
- Load the board: the client sends GET /api/tasks with Authorization: Bearer <token>; Express reads SQLite and returns JSON.
- Mutate a task: POST, PUT or DELETE persists the change and returns the resulting representation/status.
- Broadcast: after the database write, Socket.IO emits task:created, task:updated or task:deleted to all authenticated room members.
- Synchronize: each connected browser updates its task cards, counts and activity feed without reloading.

### 3. REST API design and JSON exchange

All task routes except the health check require a valid bearer token. JSON responses use one success envelope and one error envelope. IDs are positive integers; task text, status, priority and date values are validated before SQL is executed.

| Method | Endpoint           | Purpose                                                               | Success |
|--------|--------------------|-----------------------------------------------------------------------|---------|
| POST   | /api/auth/register | Validate account fields, hash password, create account and issue JWT. | 201     |
| POST   | /api/auth/login    | Verify credentials and issue JWT.                                     | 200     |
| GET    | /api/auth/me       | Resolve the token's account identity.                                 | 200     |
| GET    | /api/tasks         | List shared tasks; optional status and q filters.                     | 200     |
| POST   | /api/tasks         | Create a task and broadcast task:created.                             | 201     |
| PUT    | /api/tasks/:id     | Update task fields/status and broadcast task:updated.                 | 200     |
| DELETE | /api/tasks/:id     | Delete a task and broadcast task:deleted.                             | 200     |
| GET    | /api/health        | Check server and SQLite connectivity.                                 | 200     |

#### Example: create a task

```
POST /api/tasks
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "title": "Prepare the lab worksheet",
  "description": "Add diagrams and upload the answer key.",
  "status": "todo",
  "priority": "high",
  "dueDate": null
}
```

A successful response has HTTP 201 and includes the saved row, its generated ID, timestamps and creator attribution:

```
{ "success": true, "data": { "task": {
  "id": 24, "title": "Prepare the lab worksheet",
  "status": "todo", "priority": "high",
  "createdBy": "Alex Morgan", "creatorId": 1
} } }
```

A successful task update returns HTTP 200; deletion returns `{ "deleted": true, "id": 24 }`. Invalid fields return HTTP 400, missing resources return 404, unauthenticated requests return 401, duplicate emails return 409, and unexpected failures return 500.

## 4. Database design and real-time updates

SQLite is the persistent system of record. Prepared statements are used for reads and writes; task rows retain the account ID of their creator for attribution. The shared board returns all class tasks to authenticated accounts.

| Table | Column               | Type / rule                          | Purpose                                   |
|-------|----------------------|--------------------------------------|-------------------------------------------|
| users | id                   | INTEGER PRIMARY KEY                  | Unique account ID.                        |
| users | name, email          | TEXT; name 2-60; email UNIQUE        | Profile and sign-in identity.             |
| users | password_hash        | TEXT NOT NULL                        | bcrypt digest; plaintext is never stored. |
| tasks | id, user_id          | INTEGER; user_id FK → users          | Task ID and creator account.              |
| tasks | title, description   | TEXT; 3-120 / max 500                | Task content with database constraints.   |
| tasks | status, priority     | CHECK-constrained TEXT               | todo/doing/done; low/normal/high.         |
| tasks | due_date, timestamps | TEXT / ISO-like date; UTC timestamps | Optional due date and audit timestamps.   |

### Socket.IO event contract

The browser provides its JWT in the Socket.IO handshake. Invalid or expired tokens are rejected before the client joins `class:04-web-tech`. Server events are emitted after the REST mutation has been persisted.

| Event          | Payload fields       | Client effect                                 |
|----------------|----------------------|-----------------------------------------------|
| task:created   | task, actor, at      | Insert the new card and activity entry.       |
| task:updated   | task, actor, at      | Replace the affected card and refresh counts. |
| task:deleted   | id, title, actor, at | Remove the card and log the event.            |
| presence:count | count                | Update active classroom connection count.     |

```
{ "task": { "id": 24, "title": "Prepare the lab worksheet",
  "status": "todo", "priority": "high" },
  "actor": "Alex Morgan", "at": "2026-10-07T09:32:20.965Z" }
```

*The activity feed in the browser reflects live events in memory; it is not a separate persistent audit-log table in this demo.*

## 5. Authentication, security and error handling

| Control           | Implementation in PulseBoard                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|
| Password handling | bcryptjs hash with work factor 10 at registration; compare hash during login.                                         |
| Session           | JWT signed by JWT_SECRET with an eight-hour expiry; bearer header on REST and handshake token on Socket.IO.           |
| Request hardening | Helmet security headers; body size limited to 32 KB; authentication routes rate-limited.                              |
| Validation        | Email syntax, name/password bounds, title/description lengths, allowed status/priority, date format and positive IDs. |
| Database safety   | Parameterized SQL, foreign keys, schema CHECK constraints, and token subject resolution to a registered user.         |
| Browser output    | User-entered titles/descriptions are assigned with textContent; HTML injection is avoided.                            |
| Failure behavior  | Central JSON error envelope, meaningful 400/401/404/409/500 status codes and readable UI toast/form errors.           |

### Error response examples

```
{ "success": false, "error": {
  "code": "VALIDATION_ERROR",
  "message": "Title must be 3-120 characters."
} }
```

### Operational note

Set a long, random JWT\_SECRET before deployment. The built-in fallback is for local teaching use only. HTTPS, secure secret storage, membership-aware class rooms, stronger account recovery, audit persistence, backups, and Socket.IO scaling are intentionally outside this single-process assignment demo.

## 6. Verification results and rubric mapping

The Node.js integration suite starts an isolated HTTP server and exercises the application over HTTP and a genuine Socket.IO websocket. The final run passed all four test groups.

| Test group             | Coverage                                                                                     | Result |
|------------------------|----------------------------------------------------------------------------------------------|--------|
| Health + storage       | HTTP health response and SQLite probe.                                                       | PASS   |
| Auth                   | Register/login; bearer-token protection; wrong-password rejection.                           | PASS   |
| REST CRUD + validation | GET/POST/PUT/DELETE, filters, malformed JSON, field validation and 404 behavior.             | PASS   |
| Shared collaboration   | Cross-account board access/edit and a task event delivered to another authenticated account. | PASS   |

**Command:** ``npm test`` · **Final run:** 4 passed, 0 failed.

### Assessment rubric alignment

| Criterion                                | Maximum | Project evidence                                                                                       |
|------------------------------------------|---------|--------------------------------------------------------------------------------------------------------|
| System design & architecture             | 8       | Architecture figure, request/event workflow, component responsibilities and documented API routes.     |
| RESTful web service                      | 14      | Authenticated GET/POST/PUT/DELETE task endpoints, filters, HTTP status codes and JSON error contract.  |
| Database & JSON exchange                 | 10      | SQLite users/tasks schema, constraints, persistence, creator attribution and JSON examples.            |
| Advanced technologies, security & errors | 8       | JWT + bcrypt, Socket.IO room broadcasts, validation, Helmet, rate limiting and central error handling. |
| Total                                    | 40      | This table maps evidence to criteria; it does not award a grade.                                       |

### Known limitations

The app models one shared classroom and one server process. Users who can register join the same room; authorization is therefore suitable for a controlled classroom demonstration, not a public multi-class service. The report screenshots show a real task created from a second signed-in account and delivered to the first account.

## 7. Application screenshots

The following figures were captured from the running web application. The live board image shows the shared classroom UI after a second authenticated account creates a task; the first browser displays the activity update and task card without reloading.

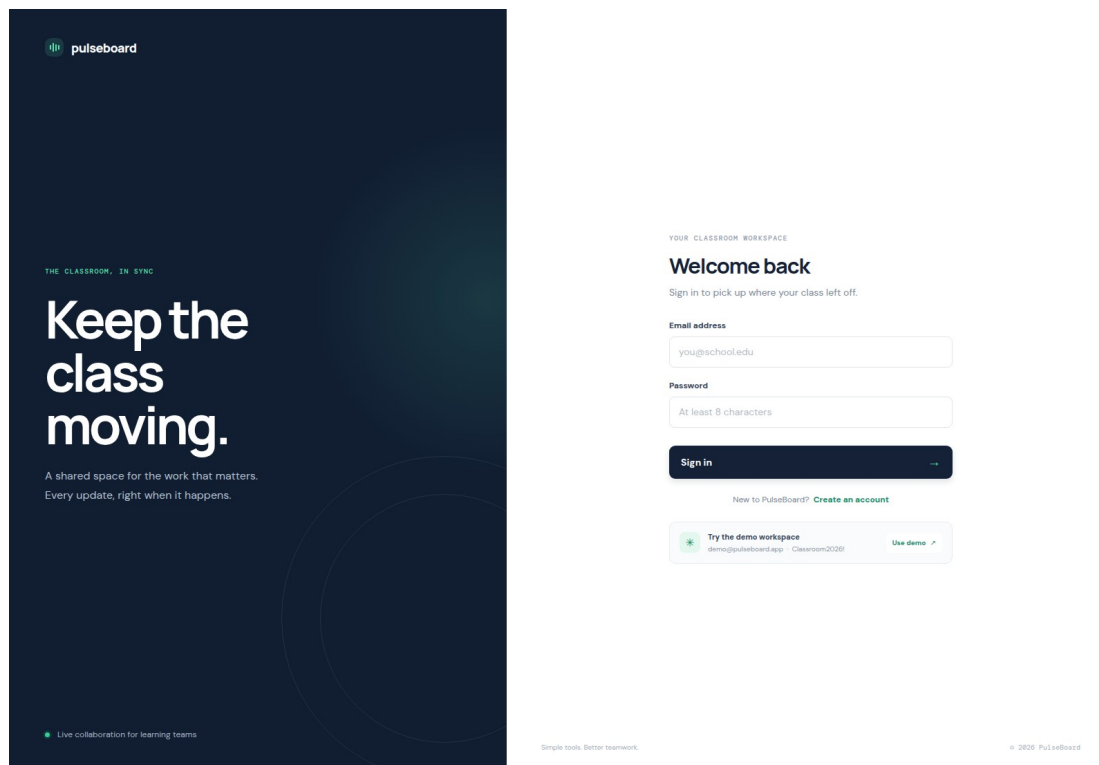


Figure 2. Sign-in screen with account creation and the local demo access option.

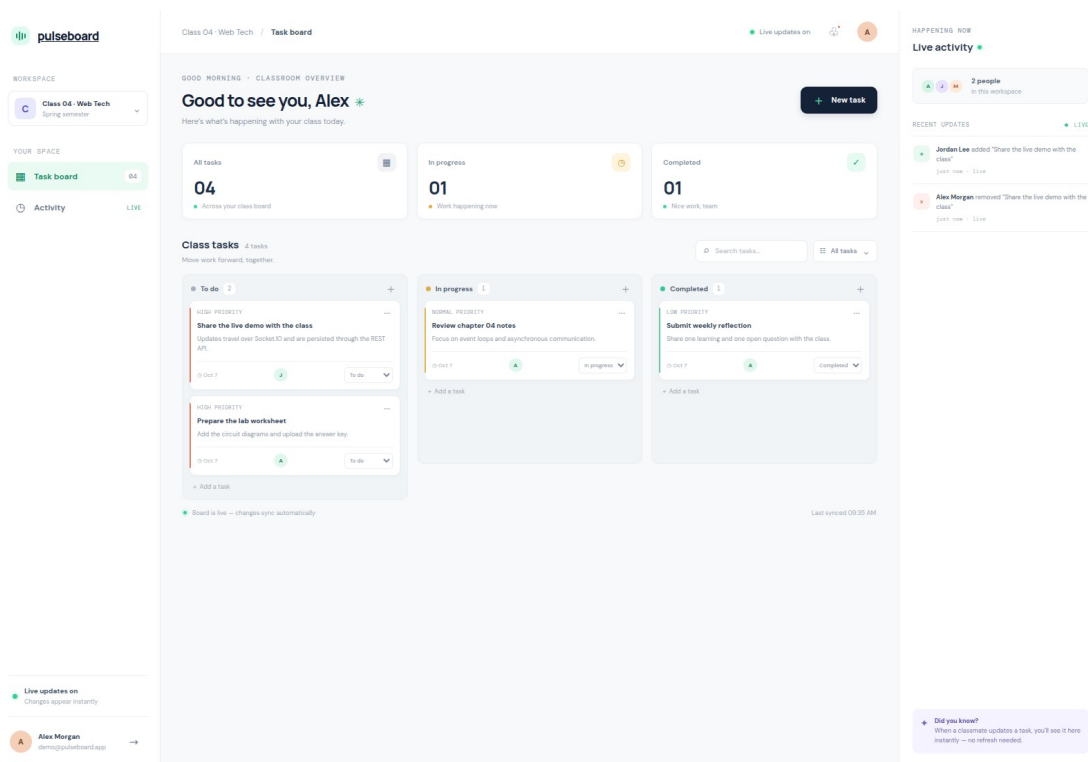


Figure 3. Shared classroom board with REST-backed task lanes and a Socket.IO activity update from another account.



## 8. Run instructions, project structure and conclusion

### Run locally

```
Requirements: Node.js 20+ and npm

npm install
export JWT_SECRET="replace-with-a-long-random-secret"
npm start

Open http://localhost:3000
Demo: demo@pulseboard.app / Classroom2026!
```

The database is created automatically at `data/pulseboard.sqlite`; first startup creates the demo account and three sample tasks. Run `npm test` to execute the integration suite.

### Project structure

| Path                                    | Role                                                                               |
|-----------------------------------------|------------------------------------------------------------------------------------|
| server.js                               | REST endpoints, JWT middleware, Socket.IO room, static hosting and error handling. |
| src/database.js                         | SQLite schema, initialization, seed data and database connection.                  |
| public/index.html · styles.css · app.js | UI, responsive styles, fetch client, CRUD and socket synchronization.              |
| test/api.test.js                        | HTTP/WebSocket integration tests.                                                  |
| docs/                                   | Assignment report, screenshots, diagram and report builder.                        |
| README.md · plan.md                     | Setup, architecture notes and implementation plan.                                 |

### Conclusion

PulseBoard demonstrates the full request-and-push cycle expected from a real-time web application: authenticated JSON requests persist shared classroom tasks in SQLite, and authorized WebSocket clients receive the committed change immediately. The implementation covers architecture, CRUD, data handling, authentication, validation and error feedback, with automated integration evidence and screenshots included for assessment.

**Deliverables:** runnable source in the [WEB\\_TECH](#) repository, [this DOCX report](#), and [its PDF export](#).